

Project Name: Synapse

1. Brief Description

Synapse is a dynamic, AI-powered codebase visualization and exploration tool. It acts as an interactive architectural map and an intelligent repository mentor, allowing developers to query a codebase as a unified context (similar to NotebookLM) while automatically generating and updating visual logic trees in real-time as Pull Requests are merged.

2. Core Features

- **Visual Repository Logic Flow:** Dynamically generates architecture diagrams and logic trees that represent how data and functions flow across multiple files.
- **Context-Aware Codebase Q&A:** Operates strictly within the context of the ingested repository, providing highly specific answers to developer inquiries regarding system mechanics without hallucinating external libraries.
- **Real-Time CI/CD & PR Monitoring:** Actively listens to GitHub webhooks. Upon a successful PR merge, it isolates the differences and updates the visual logic tree and vector embeddings continuously without requiring a full repository reindex.

3. The Problem Landscape

The modern software engineering ecosystem is plagued by an escalating crisis in codebase comprehensibility and developer onboarding. As enterprise and open-source repositories grow into monoliths or complex microservice networks, the connective tissue between files, functions, and logic flows becomes obscured. The primary friction points include:

- **The Onboarding Time-Sink:** Junior developers and new open-source contributors spend weeks tracing undocumented code execution paths manually, leading to delayed productivity and high abandonment rates in open-source projects.
- **Stale Documentation:** Traditional static architecture maps (like UML diagrams) become obsolete the moment a new feature is merged. Maintaining these maps manually is a high-friction, low-reward task for engineers.
- **Limitations of Traditional LLMs:** Standard code-completion AI lacks global repository context. While an LLM can explain a single function, it fails to understand how a change in a database schema file cascades through an abstract syntax tree to affect the frontend.

4. The Synapse Solution

Synapse solves the codebase comprehension crisis by transforming static, sprawling text into a living, visual entity. By combining advanced Retrieval-Augmented Generation (RAG) with interactive frontend visualization, Synapse delivers a multi-dimensional understanding of complex software.

When a developer queries Synapse, the system does not merely return text; it highlights the exact execution path on a dynamically generated flow graph. Furthermore, Synapse integrates directly into the development lifecycle. By hooking into GitHub, Synapse ensures that the visual architecture is always a 1:1 reflection of the main branch, eliminating the "stale documentation" problem entirely. It provides a localized, expert-level mentor for every unique codebase in existence.

5. Technical Stack & Architecture

Component	Technology / Tool	Purpose
Frontend	Next.js, React Flow (or D3.js)	Delivers the UI and renders the interactive logic map.
Backend API	Python, FastAPI	Handles webhook ingestion, coordinates the AI models, and serves data to the frontend.
AI Orchestration	LangChain / LlamaIndex	Manages the Codebase RAG pipeline, AST chunking, and prompt engineering.
Vector Database	ChromaDB	Stores the semantic embeddings of the codebase for rapid retrieval.

Component	Technology / Tool	Purpose
Integration	GitHub Webhooks	Triggers updates on PR merge events to isolate differences.

6. Example Webhook Flow (Real-Time Monitoring)

1. GitHub detects the PR merged into 'main.'
2. GitHub webhook fires a POST request to the Synapse FastAPI endpoint.
3. FastAPI's payload extracts the 'commits' array and fetches 'git diff.'
4. LangChain processes ONLY the changed files (AST parsing).
5. Vector DB updates existing embeddings (Upsert operation).
6. Websocket/SSE pushes an updated JSON graph to the Next.js frontend.